UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ingeniería

Manual técnico de desarrollo

Proyecto final de Computación Gráfica e IHC

David Díaz Antúnez
Jonhatan Alexis Martínez Ojeda
Venancio Miguel Tapia Verástegui

Semestre 2027-1 | 7 de noviembre de 2026
Estado documentado: candidato Release `v0.4.0-rc.1`, commit `7ece353`.

Índice

1. Objetivo, alcance y arquitectura **2**

2. Geometría, mundos y estilo **2**

2.1. Escenario 3D y jerarquía 2

2.2. Ambientación, especies y planetas 2

2.3. Circuito, lanzamiento y cuerpos celestes 2

3. Texturas, materiales y avatar **5**

4. Cuatro recorridos y cambio de cámaras **5**

5. Iluminación y cielo **6**

6. Animaciones exigidas **7**

6.1. Tres acciones iniciadas por teclado 7

6.2. Tres ciclos continuos basados en funciones 7

7. Compilación, distribución y validación **7**

8. Conclusiones **7**

9. Bibliografía y fuentes **8**

10. Índice de comprobación de la rúbrica **8**

1. Objetivo, alcance y arquitectura

El programa es un ambiente interactivo 3D de viajes espaciales y especies intergalácticas. El ejecutable integra cuatro recorridos, planetas con ambientación distinta, objetos animados, iluminación dinámica y un avatar articulado. Este documento relaciona los criterios de la rúbrica con archivos, mecanismos y capturas del programa integrado. Las capturas corresponden a pruebas guardadas en `docs/capturas/`; cada una ilustra el estado indicado en su pie.

Se implementó en C++ y OpenGL 3.3, con GLFW para ventana/entrada, GLEW para funciones OpenGL, GLM para matrices y Assimp para modelos. `P09-424104230.cpp` contiene el ciclo principal y orquesta `src/game/world/`, `src/game/avatar/`, `src/scene/` y `src/ui/`. `P09-2-424104230.cpp` conserva pruebas de escena y cámara; no contiene el `main` compilado.

Subsistema	Responsabilidad
Motor OpenGL	Carga OBJ/materiales/texturas, shaders, skybox, luces y geometrías base.
Mundo	Pista, estación, cohetes, dos superficies, océano, NPC y naves rivales.
Escena/avatar	Árbol de nodos, transformaciones heredadas, Rek'Sai por piezas y caminata.
Cámaras/entrada	Cuatro rigs, selección F1–F4, mouse, transiciones y controles contextuales.
Interfaz	Indicador de modo/zona/estado y reloj del ciclo.

2. Geometría, mundos y estilo

2.1. Escenario 3D y jerarquía

La pista y la base se generan con mallas y matrices OpenGL en `src/game/world/Espacio.cpp`; los elementos de superficie y sus caminos se ensamblan en `src/game/world/Superficie.cpp`. Los OBJ se cargan mediante `src/engine/Model.cpp`. La jerarquía se implementa en `src/scene/SceneNode.cpp`: cada nodo mantiene su transformación local y la compone con la matriz del padre. La visibilidad también se hereda.

Listing 1: Transformación jerárquica, extracto de SceneNode::updateWorld.

```
world = (parent != nullptr ? parent->world : glm::mat4(1.0f)) * local();
for (const std::unique_ptr<SceneNode>& child : m_children)
    child->updateWorld();
```

La prueba integrada `probarJerarquiaEscena()` comprueba la posición heredada, búsqueda de hijos y ocultamiento de un subárbol; el registro se encuentra en `docs/capturas/ha1_jerarquia.txt`.

2.2. Ambientación, especies y planetas

El planeta rocoso usa terreno mineral, cristales, portales y fauna del Vacío; el selvático usa vegetación, camino y criaturas de referencia Shinobi. Sus NPC se colocan en claros a ambos lados del camino, con tipos y movimientos diferentes. Hay además un océano explorable. El contraste entre adaptaciones rocosas/excavadoras y criaturas del entorno selvático comunica la evolución temática mediante silueta, escala y hábitat; es una representación visual, no una simulación biológica.

`Superficie::GenerarDesierto` y `Superficie::GenerarSelva` distribuyen objetos, claros, NPC y portales; `Superficie::DibujarSuperficie` los representa. `Espacio::ConstruirBase`, `Espacio::DibujarPista` y `Espacio::DibujarCinturon` completan el entorno espacial. La estación se construye en `DibujarHangar`: disco, cúpula, balizas y Fighter_02 que sale y regresa mediante una máquina de estados, y puerta de dos hojas.

2.3. Circuito, lanzamiento y cuerpos celestes

`Espacio::ConstruirTramo`, `MallaTramo` y `PuntoEnPista` generan y muestrean el circuito. En `Jugador.cpp`, `ActualizarJugador` mantiene la nave sobre la pista y permite acelerar, frenar y cambiar de carril. `EstadoRival` calcula la posición de dos rivales según el tiempo. Hay lunas, asteroides y satélite en el espacio; el cinturón de asteroides se dibuja en `Espacio::DibujarCinturon`.



Figura 1: Cuatro especies/variantes visibles en el planeta rocoso, con texturas y entorno mineral.



Figura 2: Estación espacial y Fighter_02, observables con F4 y Tab.

El cohete existe en la base y en Shinobi. `Espacio::LanzarCohete` inicia el evento por tecla 1; `Espacio::ActualizarCohetes` recorre los estados plataforma, ignición, ascenso, órbita y reinicio. Su altura durante el ascenso depende del cuadrado del tiempo normalizado, con humo y llama asociados. El modo de observación puede enfocar ambos sitios.

Listing 2: Ascenso acelerado, extracto de Espacio::ActualizarCohetes.

```
float t = glm::clamp(c.tiempoEstado / COHETE_DURACION_ASCENSO,
                 0.0f, 1.0f);
c.altura = COHETE_ALTURA_ORBITA * t * t;
c.posicion = c.origen + Y * (ALTURA_PLATAFORMA_COHETE + c.altura);
```



Figura 3: Nave del jugador sobre el circuito espacial.



Figura 4: Cohete de Shinobi en ascenso, iniciado con 1.

3. Texturas, materiales y avatar

Los recursos visuales usan OBJ/MTL, imágenes de textura y materiales con parámetros especulares. `Model::LoadModel` resuelve rutas de texturas importadas y aplica una textura de respaldo si falta un mapa; `Texture::LoadTextureA` informa un fallo de carga. `Scene::render` configura textura, tinte, material y matrices para los nodos. El shader de iluminación recibe textura, normales, color, brillo, luz direccional, puntuales y focos. Las capturas permiten verificar el aspecto del resultado; una revisión de autoría y licencias de todos los recursos se gestiona por separado.

Rek'Sai se divide en cuerpo, cabeza, cola, cuatro patas y dos brazos. `Avatar::attach` carga la geometría por partes, crea un nodo para cada pieza, les asigna la misma textura y los añade a un nodo raíz. La forma no humanoide es la del personaje controlable. `Avatar::update` orienta la raíz según el desplazamiento y, al caminar, rota extremidades contralaterales con funciones seno; al detenerse, reinicia los ángulos.

Listing 3: Paso alternado, extracto de Avatar::update.

```
const float stride = std::sin(m_walkPhase) * 0.78f;
const float oppositeStride =
    std::sin(m_walkPhase + 3.1415926f) * 0.78f;
m_legPivots[0]->rotationEuler.x = stride;
m_legPivots[1]->rotationEuler.x = oppositeStride;
```



Figura 5: Prueba visual de Rek'Sai texturizada y del render de escena.

4. Cuatro recorridos y cambio de cámaras

`CameraDirector::setMode` selecciona un `CameraRig` propio de cada modo: `FollowXZ` para planeta, `Chase` para carrera, `Flight` para vuelo y `Orbit` para observación. La proyección usa FOV de 45 grados convertido a radianes. `Input.cpp` convierte los flancos de F1–F4 en `ControlFrame`; el `main` cambia rig, zona y objeto seguido. F5–F8 permiten saltos de zona para demostración.

Modo	Cámara	Comprobación
F1	FollowXZ	Avatar a pie en roca/selva, altura ligada al plano del suelo.
F2	Chase	Nave del usuario sobre circuito; el objetivo es la nave en movimiento.
F3	Flight	Nave en vuelo espacial, con desplazamiento vertical Q/E.
F4	Orbit	Lanzamiento, hangar, planetas, satélite y cohete de base con Tab; el jugador no se teletransporta.

El cambio correcto se comprueba en `probarCameraDirector()` y en capturas `ha4_4_f1_planeta.png` a `ha4_4_f4_observacion.png`. El cuarto recorrido usa una lista de encuadres en `P09-424104230.cpp`; `SiguienteParada` omite objetivos cuyos modelos no están disponibles.

Listing 4: Asignación de rig por modo, extracto de CameraDirector.cpp.

```
case CameraMode::Planeta: return std::make_unique<FollowXZ>();
case CameraMode::Carrera: return std::make_unique<Chase>();
case CameraMode::Vuelo: return std::make_unique<Flight>();
case CameraMode::Observacion: return std::make_unique<Orbit>();
```

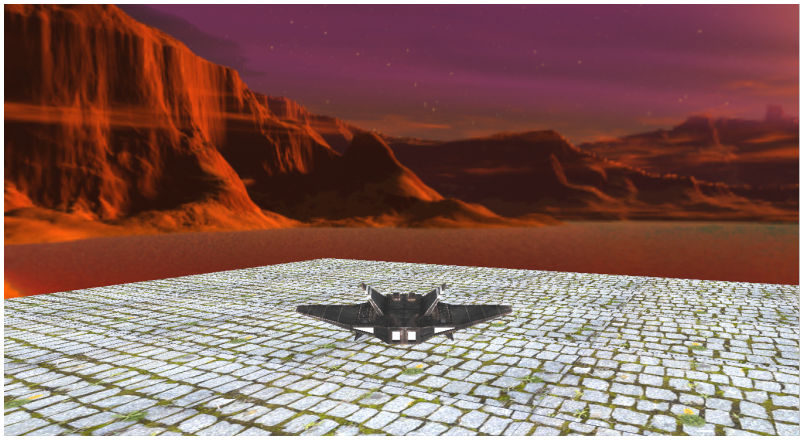


Figura 6: Prueba de la cámara de vuelo.

5. Iluminación y cielo

`ActualizarCicloDiaNoche` calcula un ciclo de 120 s. El ángulo del sol determina dirección, intensidad y color de la luz direccional; `factorDia` también gobierna el estado nocturno de las luminarias. El shader recibe las luces en cada zona. Las dos luces puntuales junto a la cápsula reciben color juntas cuando `esDeNoche`; el mismo estado modifica su emisión visual. Se usan asimismo luces puntuales de elementos activos como el cohete.

Listing 5: Ciclo solar y criterio de noche, extractos del main.

```
float anguloSol = (dosPi / duracionCicloDiaNoche) * tiempoAnimacion;
float alturaSol = sinf(anguloSol);
float factorDia = glm::clamp((alturaSol + 0.25f) / 1.25f,
                        0.08f, 1.0f);
bool esDeNoche = (factorDia < 0.45f);
```

`LoadSkybox` prepara seis caras para cada fondo. En el ciclo de dibujo se elige `Fondo::Espacio`, `Fondo::Roca` o `Fondo::Selva` según la zona visible. Los `SpotLight` de linterna, faro y plataforma se alternan con F, L y K; su posición y dirección se actualizan con la cámara o la nave.



Figura 7: Paleta y cielo propios del planeta rocoso.

6. Animaciones exigidas

6.1. Tres acciones iniciadas por teclado

1. **Cohete (1):** `Espacio::LanzarCohete` inicia ignición/ascenso; una segunda activación desde órbita devuelve el cohete.
2. **Puerta de cápsula (2):** `AnimarCapsula` interpola la apertura o cierre según su estado con la tecla 2.
3. **Paneles de satélite (3):** `AnimarSatelite` actualiza la apertura o cierre de paneles según la tecla 3.

Las tres son objetos distintos. La caminata del avatar se documenta aparte y no se cuenta en esta terna.

6.2. Tres ciclos continuos basados en funciones

1. **Naves rivales:** `Jugador::EstadoRival` calcula su avance continuo por los segmentos de la carrera según el tiempo.
2. **NPC:** `Superficie::DibujarSuperficie` aplica rebotes, ochos, elipses, levitación o respiración parametrizados con seno/coseno según la especie.
3. **Satélite en órbita:** `PosicionSatelite` orienta el satélite sobre una trayectoria de ocho sin dibujar marcas; sus paneles y rotor mantienen movimiento continuo.

También giran asteroides y corren ondas luminosas en la pista. Para la evaluación conviene mostrar tres objetos distintos en el video y señalar su movimiento sin pulsar teclas.

Listing 6: Movimiento continuo del satélite.

```
glm::vec3 satPos = PosicionSatelite(estadoSatelite, thetaSatelite);
glm::vec3 satTan = TangenteSatelite(estadoSatelite, thetaSatelite);
```

7. Compilación, distribución y validación

La solución `LabCGeIHC.sln` compila Release x64 con Visual Studio y el toolset v143. `scripts/Build.ps1` automatiza el build. `scripts/Package-Release.py` arma un ZIP con EXE, bibliotecas, shaders, modelos y texturas; incluye `release.json` con versión, commit y hashes por archivo. `Verify-Release.ps1` verifica el contenido distribuido y `Jugar.cmd --probar-release` recorre espacio, roca, selva, océano, carrera, lanzamiento y hangar. La bitácora de David registra compilación y prueba locales para `v0.4.0-rc.1`; la comprobación en otra PC debe registrarse de manera independiente antes de la entrega.

8. Conclusiones

David Díaz Antúnez. La integración de motor, cámaras y Release muestra la utilidad de separar rigs y probar los cambios en un ejecutable común. La carga diferida por zona redujo el tiempo de entrada observado; el registro de QA permite distinguir una compilación correcta de una experiencia visual comprobada.

Jonhatan Alexis Martínez Ojeda. La construcción del mundo mediante caminos, superficies y colocación parametrizada permite reutilizar mecanismos sin hacer idénticos los planetas. La coordinación de modelos, escalas y rutas de recursos fue necesaria para que las piezas funcionaran en el programa integrado.

Venancio Miguel Tapia Verástegui. El avatar articulado evidencia que separar piezas y pivotes permite animar la caminata con reglas sencillas. La entrada y el HUD ayudan a convertir funciones técnicas en recorridos que el usuario puede reconocer y controlar.

Conclusión grupal. El proyecto une escenas 3D, interacción, animación e iluminación en una experiencia compartida. La división por ramas hizo posible desarrollar partes en paralelo, pero exigió validación de integración y un paquete de recursos coherente. Cada integrante debe revisar y ratificar su conclusión individual antes de entregar este manual.

9. Bibliografía y fuentes

1. Laboratorio de Computación Gráfica e IHC, Facultad de Ingeniería, UNAM. *Lineamientos del proyecto final, grupos 2 y 3, semestre 2027-1*. PDF incluido en `docs/`.
2. Equipo CGICH. *Propuesta del proyecto final de gráfica*. PDF incluido en `docs/`.
3. Equipo CGICH. Código fuente, `README.md`, `PLAN_MAESTRO.md` y bitácoras individuales, repositorio https://github.com/Cuervy117/CG_proyecto.
4. The Khronos Group. *OpenGL 3.3 Core Specification*; documentación de GLFW, GLM, GLEW y Assimp consultada para los subsistemas correspondientes.

10. Índice de comprobación de la rúbrica

Esta tabla sirve para localizar la demostración técnica. Una captura aislada acredita la apariencia de un instante; las animaciones y los controles se verifican al ejecutar el programa y en el video demo.

Criterio del escenario	Implementación y evidencia
Geometría 3D; jerarquía	`Espacio.cpp`, `Superficie.cpp`; `SceneNode.cpp`; Figura 5 y prueba `ha1_jerarquia.txt`.
Ambientación y estilo visual	`Superficie::GenerarDesierto/GenerarSelva`, `DibujarHangar`; Figuras 1, 2 y 7.
NPC de otras especies	`Superficie::ColocarNPC` y sus variantes; Figura 1.
Pista y naves corriendo	`Espacio::ConstruirTramo`, `Jugador::EstadoRival`; Figura 3.
Plataforma y lanzamiento	`Espacio::LanzarCohete/ActualizarCohetes`; Figura 4.
Estación y entrada/salida	`DibujarHangar`; Figura 2 y cámara F4.
Luna, asteroides y satélite	`Espacio::DibujarCinturon`, dibujo espacial en el main; F4/Tab.
Dos planetas y evolución	`Superficie::GenerarDesierto/GenerarSelva`; Figuras 1 y 7; F5/F6.
Texturas de escenario	`Model.cpp`, `Texture.cpp`, `Scene.cpp`; Figuras 1–7.
Avatar no humanoide y jerárquico	`Avatar::attach`, `SceneNode::add`; Figura 5.
Material y caminata del avatar	`Scene::render`, `Avatar::update`; Figura 5 y prueba WASD.
R1: cada planeta y plano XZ	`FollowXZ.cpp`, F1, F5/F6; prueba `ha3_followxz.txt`.
R2: nave en carrera	`Chase.cpp`, F2, `Jugador.cpp`; Figura 3.
R3: vuelo entre planetas	`Flight.cpp`, F3 y Q/E; Figura 6.
R4: cohete, estación y celestes	`Orbit.cpp`, `SiguienteParada`, F4/Tab; Figuras 2 y 4.
Cambio correcto de cámaras	`CameraDirector.cpp`; prueba `ha4_4_camaras.txt`.
Luz direccional y día/noche	`ActualizarCicloDiaNoche`; Figura 7.
Puntuales sincronizadas	`pointLights`, `esDeNoche` en el main; 120 s.
Skybox variable	`LoadSkybox`, `Skybox.cpp`; Figura 7.
Spotlights por teclado	`SpotLight.cpp`, controles F/L/K; prueba interactiva.
Tres animaciones por teclado	Cohete 1, puerta 2 y paneles 3; sección 6.1.
Tres ciclos continuos	Rivales, NPC y satélite; sección 6.2.